RL Training Design for the Scientist Model

Hypothesis-Driven Search with Value of Information Rewards

Overview

This document describes the complete RL training design for a scientist model that learns to solve machine learning problems through hypothesis-driven experimentation. It covers the motivation behind each design decision, the formal setup, a concrete step-by-step walkthrough with examples, and the full reward structure.

The central goal is to train a scientist that finds better solutions faster, with fewer wasted experiments, by genuinely learning the structure of each problem rather than performing shallow local optimization.

1. Design Principles and Motivation

1.1 What We Actually Want

The ultimate objective is simple: the scientist should find better ML solutions, faster, with fewer wasted experiments. Everything else — surprise, information gain, plan divergence — is in service of this. Breaking it down:

- Exploration quality: the scientist **should cover the hypothesis space well**, not just exploit what already works.
- Structural learning: the scientist should build an accurate model of the problem as it goes — insights that generalize across all future choices.
- Efficient credit assignment: discoveries should have causal explanations, not just correlations.
- Transfer within the tree: insights from early experiments should propagate to later branches.

1.2 The Depth vs Breadth Tension

A core challenge is distinguishing two failure modes when an experiment underperforms:

- Hypothesis failure — the approach is genuinely wrong for this problem.
- Implementation failure — the approach is right but was not tuned correctly.

A scientist that always pivots after one failure never knows if it abandoned a good idea too early. This is the Duhem-Quine problem: when an experiment fails, you do not know which assumption caused the failure. The design must explicitly account for this by allowing the scientist to go deep on a hypothesis before declaring it falsified.

1.3 Why Self-Reported Metrics Fail

An early design considered using self-reported confidence from the scientist model as a reward signal. This was rejected for two reasons:

- It is directly gameable: the model can learn to always report high confidence after experiments to maximize reward.
- It is not grounded: language model confidence statements are not calibrated probabilities.

All reward signals in this design are computed externally from model behavior, not from what the model says about itself. This is a hard architectural constraint throughout.

2. System Architecture

2.1 The Two Artefacts

The scientist operates with two distinct artefacts:

Artefact	Purpose and Contents
Experiment Tree	The search structure. Nodes are experiments only. Each node has a type (explore / validate / challenge), a linked hypothesis ID, the experiment description, and the result after execution.
thought.md	The scientist's living mental model. Contains active hypotheses with confidence levels, structural claims about the problem, experimental results and their interpretations, and open questions. Updated after every experiment.

Separating these two artefacts is a deliberate design choice. The tree stays tractable because it only contains experiments. The thought.md captures all reasoning and belief

updates. This separation also makes reward computation cleaner: the tree gives us the action sequence, the thought.md gives us the belief state.

2.2 Node Types

Every experiment node must be labelled with one of three types. The type is enforced as a constraint — invalid nodes are rejected and the scientist must re-propose.

Node Type	Rules and Intent
explore	Must introduce or open a new hypothesis in thought.md. Rewarded for informative hypotheses that genuinely partition the experiment space.
validate	Must link to an existing unvalidated hypothesis. Allowed until the hypothesis flips to validated=true or the per-hypothesis budget is exhausted. Goal is to confirm or cleanly falsify.
challenge	Only allowed once the linked hypothesis has validated=true. Goal is deliberate stress-testing of established beliefs.

This structure enforces a phase discipline. The scientist cannot jump to challenging a hypothesis it has not yet validated, and cannot keep validating a hypothesis it has already established. This prevents both premature abandonment and infinite confirmation-seeking.

2.3 Budget Structure

The total budget is N tree nodes, fixed upfront. Within this budget:

- Each hypothesis gets a sub-budget of validate nodes before it must be declared confirmed or abandoned.
- Challenge nodes are drawn from the global budget, not the per-hypothesis budget.
- When the per-hypothesis validate budget is exhausted without resolution, confidence is set to low and the hypothesis is marked abandoned.

This budget structure means the scientist must be strategic about how many hypotheses it pursues simultaneously. Spreading budget thin across many hypotheses means none get properly tested. This is intentional — it creates a real tradeoff that the RL signal will teach the scientist to navigate.

3. Measuring Hypothesis Quality: Value of Information

3.1 Why Novelty Alone is Insufficient

An early design used embedding-based novelty to filter hypotheses — a hypothesis was valid if it was semantically different from existing ones. This was rejected because novelty does not imply relevance. A hypothesis can be entirely novel and still add no useful information about which experiments to run.

Example	"The Fourier transform of the target variable has interesting spectral properties" is novel on a tabular classification problem. It is also completely irrelevant — knowing whether it is true or false changes nothing about what experiments to run next.

3.2 Value of Information

The right framing is Value of Information (VoI) from decision theory. A hypothesis is valuable if resolving it — learning whether it is true or false — changes what experiments the scientist would run next.

Formally, a good hypothesis concentrates the experiment distribution. It does not matter whether concentration is toward better approaches (score-improving) or away from worse approaches (space-pruning). Both are equally valuable given a fixed node budget.

Score-Improving	"Tree methods dominate on this dataset" — resolving this concentrates experiments toward XGBoost, RandomForest, LightGBM. Directly improves best score.

Space-Pruning	"RNNs will not work on this image classification task" — resolving this eliminates an entire model class from consideration. Does not improve best score but saves significant budget.

Pure score-based VoI would miss the space-pruning case entirely. The experiment-distribution-based formulation captures both.

3.3 Computing VoI

For a proposed hypothesis H, we compute how much the experiment distribution shifts when H is resolved:

```
# Current experiment distribution — scientist's plan given thought.md as-is

P_unresolved = empirical distribution over K GRPO samples



# Condition on H being true — prepend to thought.md, sample K experiments

P_H_true = empirical distribution over K conditional samples



# Condition on H being false — prepend to thought.md, sample K experiments

P_H_false = empirical distribution over K conditional samples



# Weight by current belief about H (logit-based, see Section 3.4)

p = logit_probability(model, thought_md, H)   # P(H = true | thought.md)

P_resolved = p * P_H_true + (1 - p) * P_H_false



# VoI = how much the distribution shifts upon resolution

VoI(H) = KL(P_resolved || P_unresolved)
```

The experiment space is discretised into categories (tree methods, neural nets, feature engineering, preprocessing, ensembling) to make the distribution tractable. The K samples are reused from GRPO, so VoI computation adds only two extra forward passes per tree node.

3.4 Estimating P(H = true) via Logits

The weighting of P_H_true and P_H_false requires a current belief estimate. Self-reported probability is gameable, so we extract it directly from the model's token probabilities:

```
prompt = f"""

Given the following thought.md and problem description,

is this hypothesis true?



thought.md: {thought_md}

Hypothesis: {H}



Answer: [true/false]

"""



logits = model.forward(prompt)

p_true  = softmax(logits["true"])

p_false = softmax(logits["false"])
```

The model cannot game this — it does not control its own logits, only its generated text. This also gives the correct Bayesian behaviour: if thought.md already contains strong evidence for H, the logit for true will be high, P_resolved will approach P_H_true, the KL divergence from P_unresolved will be small, and Vol will be low. The scientist will not waste a node confirming what it already knows.

Validation Note	Before deploying logit-based estimation in training, run a calibration study: sample K completions per hypothesis, compute empirical P(H=true) from samples, and measure correlation with logit-derived probability. High correlation validates the approach; low correlation indicates a need for logit recalibration.

4. Reward Structure

4.1 Three Reward Components

Component	What It Measures
R1 — Resolution	Did this experiment move confidence in its linked hypothesis? Measured by prediction sharpness and whether the prediction was correct. Computed externally from ground truth, not self-reported.
R2 — Information	Did this experiment introduce a new structural claim about the problem? Measured by VoI of the hypothesis introduced (explore nodes only).
R3 — Performance	Did the tree find a good solution? Measured by best normalised score at end of tree. Applied once at the end, not per node.

4.2 R1: Resolution Reward

Before each validate or challenge experiment, the scientist must output a sharp prediction:

```
hypothesis: "Tree methods outperform neural nets on this dataset"

prediction: "XGBoost will score > MLP by at least 0.03"

experiment: run XGBoost vs MLP with standard tuning
```

After the experiment, the outcome is checked against the prediction externally. Reward is computed using a sharpness-weighted scoring rule:

```
prediction_interval = upper - lower          # narrower = more committed

prediction_correct  = actual_score in [lower, upper]
```

```
R1 = prediction_correct / max(prediction_interval, 0.01)
```

A narrow correct prediction gets high reward. A wide vague prediction gets low reward even if technically correct. This directly penalises the failure mode of predicting vaguely to avoid being wrong. The confidence update in thought.md is also computed externally — not by asking the model how confident it is, but by whether its prediction was correct.

4.3 R2: Information Reward

For explore nodes, R2 equals the VoI of the introduced hypothesis, computed as described in Section 3. For validate and challenge nodes, R2 is zero — they are not expected to introduce new hypotheses, only to resolve existing ones.

```
if node.type == "explore":

    R2 = VoI(node.hypothesis)    # KL-based, computed from K samples

else:

    R2 = 0
```

4.4 R3: Performance Reward

Applied once at the end of the entire tree rollout:

```
baseline = simple_automl_score(problem)      # e.g. default XGBoost

R3 = (best_score_in_tree - baseline) / baseline
```

This grounding term prevents the scientist from being interesting but useless. Without R3, the scientist could explore endlessly and write rich thought.md entries while never actually improving the solution.

4.5 Combined Reward

```
R_node  = alpha * R1 + beta * R2         # per node, dense signal

R_final = gamma * R3                      # end of tree, sparse signal



# Initial values

alpha = 0.4    # reward resolution of existing hypotheses

beta  = 0.3    # reward informative new hypotheses

gamma = 0.3    # reward actual performance
```

The per-node rewards provide dense signal that makes credit assignment tractable. The final reward ensures the search stays grounded in actually solving the problem. The balance between these is a tuning question, but the starting point reflects a slight emphasis on structural learning over raw performance, on the expectation that structural learning will lead to better performance over longer horizons.

5. The GRPO Training Loop

5.1 Phase 1: SFT

Before RL training, the model is trained via supervised fine-tuning on existing trajectories of good scientists solving ML problems. This phase installs the prior: domain knowledge, what good experiments look like, how to structure thought.md, how to write hypotheses. The RL phase refines how the scientist navigates this prior — it cannot substitute for it.

5.2 Phase 2: GRPO on Tree Rollouts

For each training problem, the GRPO loop proceeds as follows:

1. Sample K=8 experiment proposals from the scientist given the current tree state and thought.md.
2. Enforce node type rules: reject proposals that violate the type constraints (e.g. challenge on an unvalidated hypothesis). Rejected proposals receive zero reward and are not included in the GRPO update.
3. For explore proposals: compute VoI for each introduced hypothesis using the K samples and logit-based P(H=true).
4. Run the selected experiment through the executor (or simulate from historical data for cheap training).
5. Update thought.md based on the result. Check the scientist's prediction against the actual outcome.
6. Compute R1 and R2 for this node.
7. Repeat steps 1–6 until the node budget N is exhausted.
8. Compute R3 on the completed tree. Assign discounted per-node returns from R3 backward through the tree.
9. GRPO update: proposals with higher cumulative reward get reinforced. The advantage is computed relative to the mean reward across the K proposals.

5.3 Credit Assignment

Credit assignment across a long tree is non-trivial. A great explore node at step 2 enables a great validate node at step 6 which enables a great challenge node at step 10 — standard discounted returns may wash out the contribution of step 2.

The per-node R1 + R2 rewards provide dense local signal that reduces dependence on R3 for credit. For the R3 component, we use standard discounted returns with a relatively high discount factor (gamma_discount ≈ 0.95) to preserve credit over long horizons. Tree-structured credit assignment (propagating credit through hypothesis linkages rather than just temporal order) is worth exploring in future iterations.

6. Concrete Worked Example

Problem: Titanic survival prediction. Budget: N = 12 nodes.

Initial State	thought.md is empty. Experiment tree is empty. The scientist reads the problem description and dataset schema.

Node 1 — explore

```
Scientist output:

  type: explore

  hypothesis H1: "Family structure is a latent variable that SibSp and Parch

                  do not capture individually — a combined FamilySize feature

                  will improve model performance."

  experiment: baseline XGBoost without FamilySize vs with FamilySize = SibSp + Parch +
1

  prediction: XGBoost + FamilySize will score > baseline by 0.02–0.04



VoI computation:

  P_unresolved: {feature_eng: 0.35, tree_methods: 0.30, preprocessing: 0.20, nn: 0.15}

  P_H1_true:    {feature_eng: 0.55, tree_methods: 0.25, preprocessing: 0.15, nn: 0.05}

  P_H1_false:   {feature_eng: 0.20, tree_methods: 0.35, preprocessing: 0.25, nn: 0.20}

  logit P(H1=true) = 0.52  (thought.md empty, roughly uncertain)

  P_resolved = 0.52 * P_H1_true + 0.48 * P_H1_false

  VoI(H1) = KL(P_resolved || P_unresolved) = 0.18   [reasonably high]



Result: XGBoost + FamilySize scores 0.793 vs baseline 0.764 — improvement of 0.029

Prediction correct (0.029 within [0.02, 0.04]).

R1 = 1.0 / (0.04 - 0.02) = 50  [normalised]
```

```
R2 = 0.18


thought.md update:

  H1: validated=false, confidence=medium

  Structural claim: FamilySize is a useful latent variable on Titanic.
```

Node 2 — validate (H1)

```
Scientist output:

  type: validate

  hypothesis: H1

  experiment: test FamilySize with LightGBM and RandomForest to check if the

              effect is model-agnostic or specific to XGBoost

  prediction: both models will show > 0.015 improvement with FamilySize


Result: LightGBM +0.021, RandomForest +0.018. Both within prediction.

R1 = 1.0 / 0.02 = 50  [normalised, sharp prediction]

R2 = 0  [validate node]


thought.md update:

  H1: validated=true, confidence=high

  Structural claim: FamilySize benefit is model-agnostic — confirms latent variable.
```

Node 3 — challenge (H1)

```
Scientist output:

  type: challenge

  hypothesis: H1  [now validated=true, challenge allowed]

  experiment: test whether more granular family features (IsAlone, LargeFamilyFlag)

              outperform simple FamilySize, which would challenge the claim that

              FamilySize captures all the latent structure

  prediction: IsAlone + LargeFamilyFlag will outperform FamilySize by < 0.005

              (expecting FamilySize to be sufficient)



Result: IsAlone + LargeFamilyFlag scores 0.801 vs FamilySize 0.793 — improvement of
0.008

Prediction incorrect (0.008 > 0.005 threshold).

R1 = 0  [prediction failed]



thought.md update:

  H1 revised: FamilySize is a useful start but granular family features

              capture additional structure. IsAlone is particularly predictive.

  New open question: Is IsAlone capturing survival-relevant social dynamics?
```

Node 4 — explore (new hypothesis from challenge result)

```
Scientist output:

  type: explore

  hypothesis H2: "Passengers traveling alone had different survival rates due to
```

```
                    social dynamics — IsAlone interacts with Pclass and Sex."

  experiment: add IsAlone * Pclass and IsAlone * Sex interaction terms

  prediction: interaction features will improve score by 0.005–0.015



VoI(H2) = 0.22  [high — resolving this changes preprocessing vs feature-engineering
focus]



Result: +0.009. Within prediction.

R1 = 1.0 / 0.01 = 100 [normalised]

R2 = 0.22
```

... The tree continues for remaining 8 nodes. The scientist explores class-based imputation of Age (H3), tests whether stacking outperforms single models (H4), and challenges H3 by testing median vs model-based imputation. Each node is tied to an explicit hypothesis with a sharp prediction.

End of Tree	Best score: 0.821. Baseline: 0.764. R3 = (0.821 - 0.764) / 0.764 = 0.075. Total reward = cumulative R1+R2 across nodes + gamma * R3.

7. Known Failure Modes and Mitigations

Failure Mode	Mitigation
Model learns to make predictions just outside the interval to avoid correctness checks	Track calibration across the full tree. Penalise systematic bias (always too high or too low) in R3.

VoI computation is expensive at scale	Reuse K GRPO samples for P_unresolved. Only two extra forward passes per node for P_H_true and P_H_false.
Logit estimates are poorly calibrated	Run a calibration study correlating logit-based P(H=true) with sampling-based estimates before deploying in training.
Depth vs breadth: model spreads budget too thin	Per-hypothesis validate budget enforces commitment. Model must go deep or abandon, not hover at medium confidence indefinitely.
Challenge nodes used to game the type constraint	Challenge only allowed after validated=true, and the linked hypothesis is checked at reward time.
Credit assignment washes out early good decisions	Dense per-node R1+R2 rewards reduce dependence on sparse R3 signal. High discount factor preserves long-range credit.

8. Summary of Design Decisions

Decision	Rationale
Tree over experiments only, not hypotheses	Keeps the search space tractable. Hypotheses live in thought.md where they can be tracked without exponentially expanding the tree.
Node types enforced as hard constraints	Creates phase discipline. Prevents premature abandonment, infinite confirmation-seeking, and unconstrained exploration.
VoI over novelty for hypothesis quality	Novelty checks whether a hypothesis is different. VoI checks whether it matters. Relevance and novelty are independent — VoI measures both together.
Logit-based belief estimation	Non-gameable, cheap (single forward pass), and gives correct Bayesian behaviour: known hypotheses get low VoI automatically.
Sharpness-weighted prediction scoring for R1	Penalises vague predictions. The model cannot hedge by giving wide intervals — it must commit and be checked against reality.
All reward computed externally	The model has no control over the reward signal. Eliminates self-reporting failure modes across the board.
Dense per-node rewards + sparse end reward	Dense R1+R2 make credit assignment tractable. Sparse R3 ensures the search stays grounded in solving the problem.